The middle term is bounded by δ_t^{arith} by (8). The first term is bounded by $L_t\|\widehat{\lambda}_t - \lambda_t\|_\infty \le L_t\delta_t^{\text{route}}$ using (9) (absorbing the realized-vs-ideal dependence into the same sensitivity bound). The last term is bounded by $K_t\|\widehat{s}_t - s_t\|_\infty$ by (10). Combining yields the claim. □

Remark (state Lipschitz constant). The constant K_t in (10) depends on the ideal update Ψ_t and the domain. If one prefers to keep the bookkeeping additive, it suffices to enforce $K_t \le 1$. In our executor design this can be achieved (when needed) by scaling and residual structure, since the work-state typically enters through residual additions and coordinate-wise bounded operations.

E.4. Multi-step composition and a generic budget split

Lemma E.2 (End-to-end bound by summing local budgets)**.** *Assume the hypotheses of Lemma E.1 for all* $t = 0, \dots, T_{\text{macro}} - 1$ *and* $\widehat{s}_0 = s_0$*. Then for any* $(p, x) \in P \times X$*,*

$$\|\widehat{s}_{T_{\text{macro}}} - s_{T_{\text{macro}}}\|_\infty \;\le\; \sum_{t=0}^{T_{\text{macro}}-1} \left(\delta_t^{\text{arith}} + L_t\,\delta_t^{\text{route}}\right).$$

Proof. Apply Lemma E.1 iteratively and telescope using $\widehat{s}_0 = s_0$. □

E.5. Instantiating the template

A convenient uniform allocation is to choose per-step budgets

$$\delta_t^{\text{arith}} \equiv \frac{\varepsilon_{\text{exec}}}{2T_{\text{macro}}}, \qquad L_t\,\delta_t^{\text{route}} \equiv \frac{\varepsilon_{\text{exec}}}{2T_{\text{macro}}},$$

so that Lemma E.2 yields $\|\widehat{s}_{T_{\text{macro}}} - s_{T_{\text{macro}}}\|_\infty \le \varepsilon_{\text{exec}}$.

From executor state error to function error. Let the executor output be a scalar readout $F_{\theta^*}(p, x)$ from a designated token as in Appendix B. Since the readout is affine in that token, there exists a constant C_{out} (e.g., $\|v_{\text{out}}\|_1$) such that

$$|F_{\theta^*}(p, x) - \widetilde{F}_{\theta^*}(p, x)| \;\le\; C_{\text{out}}\,\|\widehat{s}_{T_{\text{macro}}} - s_{T_{\text{macro}}}\|_\infty,$$

where $\widetilde{F}_{\theta^*}$ denotes the idealized output under perfect routing/arithmetic at the macro level. Thus, allocating budgets to make the final state error at most $\varepsilon_{\text{exec}}/C_{\text{out}}$ yields an end-to-end executor error at most $\varepsilon_{\text{exec}}$ uniformly over $x \in X$.

Connecting to the main theorem. In the overall proof, we first choose an MLP N_g so that $\|N_g - g\|_\infty \le \varepsilon_{\text{approx}}$. Then we design a prompt p_g and fixed executor θ^* so that the executor emulates N_g with uniform error at most $\varepsilon_{\text{exec}}$. By the triangle inequality,

$$\sup_{x \in X}\left|F_{\theta^*}(p_g, x) - g(x)\right| \;\le\; \varepsilon_{\text{exec}} + \varepsilon_{\text{approx}} \;\le\; \varepsilon.$$

F. Prompt Encoding as Key–Value Memory

This appendix makes the "prompt as memory" viewpoint explicit and records a concrete encoding scheme. For a prompt matrix $P = [p_1; \dots; p_L] \in \mathbb{R}^{L \times D}$, define

$$K_P := PW_K, \qquad V_P := PW_V,$$

where $W_K, W_V \in \mathbb{R}^{D \times D}$ are the same matrices used inside the attention module $\mathcal{A}_\tau$. Given a query $q \in \mathbb{R}^D$, the prompt read operation is

$$\mathcal{R}_\tau(q; P) \;:=\; S_\tau(qK_P^\top)\,V_P \in \mathbb{R}^D,$$

where S_τ is the row-wise softmax with temperature $\tau > 0$.

F.1. Memory layout: parameter slots and work-memory slots

In our constructions, we partition the L prompt slots into parameter storage and work memory:

$$[L] \;=\; \mathcal{I}_{\mathrm{par}} \,\dot\cup\, \mathcal{I}_{\mathrm{wrk}}. \tag{11}$$

Slots in $\mathcal{I}_{\mathrm{par}}$ store task-specific records (parameters/control signals) that the executor retrieves via $\mathcal{R}_\tau(\cdot; P)$. Slots in $\mathcal{I}_{\mathrm{wrk}}$ are optional scratch memory used by some variants (e.g., to cache intermediate values). In the simplest instantiation one may take $\mathcal{I}_{\mathrm{wrk}} = \emptyset$.

For each slot $j \in [L]$, we denote its projected key and value by

$$k_j := (K_P)_{j,:} \in \mathbb{R}^D, \qquad v_j := (V_P)_{j,:} \in \mathbb{R}^D.$$

Then $\mathcal{R}_\tau(q; P)$ is a convex combination of the value vectors $\{v_j\}_{j=1}^L$ with weights determined by the key similarities $\{\langle q, k_j\rangle\}_{j=1}^L$.

F.2. Address labels and a fixed key codebook

Let $\mathcal{L}$ be a finite set of *address labels* indexing the records that may be retrieved by the executor (e.g., "row i of a matrix used at macro-step t", "bias term", etc.). Fix a key dimension $d_k \le D$ and a codebook of key vectors $\{\kappa(\ell) \in \mathbb{R}^{d_k} : \ell \in \mathcal{L}\}$ such that there exists a separation margin $\gamma \in (0, 1]$ with

$$\langle \kappa(\ell), \kappa(\ell)\rangle = 1, \qquad \langle \kappa(\ell), \kappa(\ell')\rangle \le 1 - \gamma \quad \text{for all } \ell \ne \ell'. \tag{12}$$

For finite $\mathcal{L}$, such a codebook exists whenever d_k is sufficiently large (e.g., using nearly-orthogonal vectors). The executor will generate a query $q(\ell)$ whose *key block* aligns with $\kappa(\ell)$, so that the score margin required by the routing lemma (Appendix C) holds.

Concrete choice (basis keys). A particularly simple choice is to take $\mathcal{L} \subseteq [d_k]$ and set $\kappa(\ell) = e_\ell$, in which case (12) holds with $\gamma = 1$.

F.3. Encoding labeled records into parameter slots

A record consists of a label $\ell \in \mathcal{L}$ and a payload vector $u_\ell \in \mathbb{R}^D$ (packed parameters). We implement a dictionary

$$\mathcal{M}(p) \;:=\; \{(\ell, u_\ell) : \ell \in \mathcal{L}_{\mathrm{used}} \subseteq \mathcal{L}\}$$

by assigning one parameter slot $j(\ell) \in \mathcal{I}_{\mathrm{par}}$ per label ℓ and enforcing

$$k_{j(\ell)} \approx k(\ell), \qquad v_{j(\ell)} \approx u_\ell. \tag{13}$$

The approximation can be exact in the simplified executor by choosing W_K, W_V with a block structure that separates key-coordinates from value-coordinates (so the prompt token can independently control $k_{j(\ell)}$ and $v_{j(\ell)}$), or more generally by choosing prompt tokens $p_{j(\ell)}$ to satisfy the linear constraints induced by W_K, W_V on the relevant subspace.

F.4. Packing structured parameters into payload vectors

Most payloads are structured (scalars, vectors, matrix rows/blocks). We use fixed linear packing/unpacking maps

$$\mathrm{pack} : \mathbb{R}^r \to \mathbb{R}^D, \qquad \mathrm{unpack} : \mathbb{R}^D \to \mathbb{R}^r,$$

implemented inside the executor (e.g., via affine maps realizable by token-wise FFNs). For example, a scalar $\alpha \in \mathbb{R}$ may be packed as $\mathrm{pack}(\alpha) = \alpha e_1$, and a matrix $U \in \mathbb{R}^{a\times b}$ may be stored row-wise by packing each $U_{i,:}$ into a separate payload slot.

F.5. Retrieval guarantee (record level)

The following corollary is a direct specialization of the routing bounds in Appendix C.

Corollary F.1 (Record retrieval from parameter slots)**.** *Fix $\tau > 0$. Consider a prompt dictionary stored in slots $\{j(\ell) \in \mathcal{I}_{\mathrm{par}}\}$ with keys $\{k_{j(\ell)}\}$. Suppose that for some label ℓ^* and query $q = q(\ell^*)$ we have the score margin*

$$\langle q, k_{j(\ell^*)} \rangle \;\geq\; \langle q, k_{j(\ell)} \rangle + \Delta \qquad \textit{for all } \ell \neq \ell^* \textit{ stored in } \mathcal{M}(p).$$

If all stored values satisfy $\|v_{j(\ell)}\|_\infty \leq B$, then

$$\left\| \mathcal{R}_\tau(q; P) - v_{j(\ell^*)} \right\|_\infty \;\leq\; 2B\,(L-1)e^{-\Delta/\tau}.$$

Proof. Apply Lemma C.2 to the L prompt slots, with $j^* = j(\ell^*)$. □

Remark (fixed versus prompt-dependent objects). The prompt P (hence the stored keys/values) is task-dependent. All weights in the executor, including (W_Q, W_K, W_V), the query-generation mechanism, and the fixed packing/unpacking maps, are shared and belong to θ^*.

G. Executor Program: Macro-step Layout and Invariants

This appendix describes the executor as a finite "program" executed over a sequence of macro-steps. Our goals are: (i) to make explicit which subroutines are performed at each step (routing reads, arithmetic updates, optional write-backs), and (ii) to state a small set of invariants maintained throughout execution. These invariants act as the bookkeeping interface between the macro-step description and the error analysis: they specify what must hold at each step so that routing and token-wise computation can be composed with a controlled end-to-end error.

Admissible prompts (uniformity domain). Throughout Appendix G, we restrict attention to the *admissible* prompt set $\mathcal{P}_{\mathrm{adm}} \subseteq \mathbb{R}^{L \times D}$ produced by the encoding scheme of Section 4.2 and Appendix F. In particular, prompts in $\mathcal{P}_{\mathrm{adm}}$ store records in fixed bounded ranges determined by the construction. All invariants and bounds in this appendix are uniform over $\mathcal{P}_{\mathrm{adm}}$ and $x \in \mathcal{X}$. See Remark 3.5 for the relation between the ambient prompt space $\mathcal{P}$ and the admissible subset $\mathcal{P}_{\mathrm{adm}}$ used for uniform bounds.

G.1. Registers and designated tokens

We consider a fixed set of designated token indices:

$$\mathsf{PROMPT} := \{1, \ldots, L\}, \qquad i_x := L+1, \qquad i_{\mathrm{wrk}} := L+2, \qquad i_{\mathrm{out}} := L+3,$$

so the total sequence length is $n = L + 3$. The prompt occupies PROMPT, the input token is at i_x, the *work token* is at i_{wrk}, and the *output token* is at i_{out}.

We view the work token state $z_{i_{\mathrm{wrk}}}^{(t)} \in \mathbb{R}^D$ as a register file. Fix linear projections

$$\Pi_{\mathrm{st}} : \mathbb{R}^D \to \mathbb{R}^m, \qquad \Pi_{\mathrm{qry}} : \mathbb{R}^D \to \mathbb{R}^D, \qquad \Pi_{\mathrm{tmp}} : \mathbb{R}^D \to \mathbb{R}^{m_{\mathrm{tmp}}},$$

implemented by fixed affine maps in the executor (Appendix D, Lemma D.4). We interpret

$$s_t := \Pi_{\mathrm{st}}\big(z_{i_{\mathrm{wrk}}}^{(t)}\big) \in \mathbb{R}^m$$

as the *macro-state register* at macro-step t, and

$$q_t := \Pi_{\mathrm{qry}}\big(z_{i_{\mathrm{wrk}}}^{(t)}\big) \in \mathbb{R}^D$$

as the *query register* used for prompt reads. Temporary registers Π_{tmp} are used for assembling payload fragments.

G.2. Macro-step layout

We organize the depth-T Transformer into T_{macro} macro-steps. Each macro-step consists of the following fixed pattern of subroutines: